

E07 · E-경로 — 모델 밖에적어두기

이 권은 본문 ch13(RAG에서 MemGPT로), ch14(Sleep-time Compute), ch15(Mem0·Zep·ReasoningBank)를 수식과 증명없이 다시 설명한다. 세 장이 다루는 것은 하나의 계보다 — 모델을 건드리지 않고 모델 밖에 글자를 적어 두는 방식으로 기억을 만드는 시도들이다.

이 계보에는 다른 데서 보기 어려운 사건이 둘 있고, 이 권은 그 둘을 향해간다. 첫째, 이 라인에 이름을 준 논문이 비용을 하나도 보고하지 않는다. 둘째, 여러 논문의 자기 표에서 품질 1위가 반복해서 “기억시스템 없음”이다. 둘을 합치면 결론이 하나 나온다 — 이 경로에서는 비용결과가 성능 결과의 자리에 놓여 팔리고 있다. 이 권은 그 문장을 수치로지탱하는데 절반을 쓴다.

1. 세 층 중에서 가장 만만한층

이 라인의 모든 논문은 “질의가 오기 전에 계산을 미리 해서 상태를 바꿔둔다”는 한 문장으로 환원된다. 어려운 것은 “상태”가 하나가 아니라 셋이라는 사실이다.

기호 풀이 Θ (slow weights) = 배포되는 가중치. 우리가 .safetensors로 굽고 HBM에 올리는 바로 그것이다. W (fast weights) = 요청을 처리하는 동안만 살아 있는 고정 크기 상태. KV cache와 달리 길이에 따라 자라지 않고, 새 항목을 받으면 옛항목 위에 겹쳐 쓴다. E (external store) = 모델 밖의 텍스트·벡터·그래프 저장소. Postgres 행이나 벡터 index가 여기 있다.

세 층은 크기가 아니라 누가 소유하고 언제 사라지는가로 갈린다. W 는 요청이 끝나면 사라지고, Θ 는 고치는 순간 배포 아티팩트가 바뀌며, E 는 고쳐도 서빙되는 모델이 그대로다. 그래서 셋 중 손대기가 가장 쉬운 층이 E 이고, 이 권의 논문들은 전부 E 만 건드린다. 이것을 E-경로라고 부른다.

읽기는 한 줄이다.

$$\hat{y} = f(q; \Theta, W, \text{ret}(E, q)) \quad (\text{R})$$

기호 풀이 q = 사용자 질의. E-경로에서 q 는 벡터가 아니라 토큰문자열이다. $\hat{y}$ = 모델의 답. f = 읽기 사상, 즉 한 번의 추론. $\text{ret}(E, q)$ = 저장소에서 질의에 맞는 것을 꺼내는 검색 연산.

이 식이 말하는 것은 답이 세 가지 재료로 만들어진다는 것뿐이다 — 배포 가중치, 요청 상태, 그리고 저장소에서 꺼내 온 것. E-경로는 이 중셋째 항만 건드리고 앞의 둘은 손대지 않는다.

쓰기도 한 줄이다.

$$E_{k+1} = \text{wr}(E_k, \hat{c}_k), \quad \hat{c}_k = S(c_k; B_s) \quad (\text{U-E})$$

기호 풀이 $k = \text{sleep 라운드}$ 첨자. 오프라인 정리 작업의 회차이며, 토큰첨자 t 와는 완전히 다른 시계다. $c_k =$ 라운드 k 가 받은 원문맥. $\hat{c}_k = \text{learned context}$. 질의를 보기 전에 미리수행한 추론의 산출물. $S(\cdot; B_s) =$ 예산 B_s 를 써서 문맥을 오프라인 처리하는 절차. $wr =$ 그 결과를 저장소에 반영하는 연산.

이 식이 말하는 것은 두 단계라는 것이다. 먼저 미리 생각해서 $\hat{c}_k$ 를 만들고, 그 다음 그것을 저장소에 넣는다. 이 권의 논문들이 갈리는 지점은 전부 S 의 모양과 wr 의 모양이다.

비용은 언제나 네 값으로 쟀다.

기호 풀이 $B_s = \text{sleep 계산 예산 (FLOPs 또는 tokens)}$. 미리 태우는 양이다. $L_w =$ wake 지연 기여 (ms, TTFT/ITL). 사용자가 실제로 기다리는 시간이다. $C_{\text{cap}} =$ 상태 용량 (bytes). 남겨 두는 양이다. $\rho =$ 망각 · 열화율. 라운드를 반복했을 때 라운드당 잃는 성능이다.

이 네 칸이 이 권의 채점표다. 앞으로 나오는 모든 논문을 이 표로만 읽는다.
자세한 것은 ch11 § 11.2, ch13 § 13.3.

2. RAG에는 쓰는 사람이 없었다

E-경로의 출발점을 정확히 잡으려면 RAG를 한 문장으로 정리해야 한다. RAG의 E 는 corpus 당 하나이고, 읽기 전용이며, 평평하다. 검색을 촉발하는 것은 언제나 질의이고, 최적화 대상은 $\text{ret}(E, q)$ 의 정확도다.

여기서 빠진 것이 하나 있다. RAG 논문 자신이 자기 기억을 “human-readable”이고 “human-writable”이라고 자랑하는데, 세 번째 성질인 **machine-writable**이 없다. 사람이 corpus를 만들어 넣어 주는 동안 모델은 읽기만 한다. 승격도 강등도 삭제도 연산으로 존재하지 않는다.

E-경로가 시작되는 자리는 저장소가 커진 자리가 아니라 **저장소를 운영하는 주체가 처음으로 모델이 된 자리**다.

자세한 것은 ch10 § 10.2, ch13 § 13.1.

3. MemGPT — 프롬프트를 main memory라고 부르기

MemGPT(arXiv:2310.08560)는 문제를 이렇게 세운다. 문맥창을 키우는 것은 답이 아니다. 창을 키워도 모델이 가운데를 제대로 쓰지 못한다는 관측이 이미 있으므로, 고정 창을 그대로 두고 **무한 문맥의 착시**를 만들자는 것이다.

방법은 저장소를 두 층으로 가르는 것이다. 프롬프트 **안**에 사는 층과 프롬프트 **밖**에 사는 층을 나누고, 둘 사이의 이동을 명시적 연산으로 관리한다. 프롬프트 안에는 셋이 있다 — 함수 스키마를 담은 읽기 전용 정적 블록, 모델이 직접 고쳐 쓸 수 있는 고정 크기 텍스트 블록 (working context), 그리고 롤링 메시지 큐. 프롬프트 밖에는 둘이 있다 — 들어오고 나간 모든 메시지를 담은 recall storage와, 길이 제한 없는 텍스트를 담은 archival storage다.

여기서 이 논문의 결정적 전환이 나온다. **이동을 촉발하는 것이 질의가 아니라 프롬프트 길이다**. 프롬프트가 문맥창의 70%를 넘으면 “memory pressure”라는 경고 문자열이 큐

에 붙고, 100%를 넘으면 창 의 약 50%를 잘라 내면서 잘린 부분을 요약해 앞자리에 밀어 넣는다. RAG 의 E 설계에서 1 차 변수는 검색 품질이었지만, 여기서는 어느 층에 무엇을 남길 것인가로 바뀐다.

비유 RAG 는 캐시 적중률을 올리는 논문이고 MemGPT 는 캐시 교체 정책을 만드는 논문이다. — **비유가 깨지는 곳**: 진짜 캐시의 교체 정책은 하드웨어나 런타임이 강제로 실행하지만, MemGPT 의 교체 정책은 **모델 자신이 텍스트로 발화해야** 실행된다. 모델이 발화하지 않으면 아무 일도 일어나지 않는다.

이 균열이 논문이 스스로 붙인 이름에서 그대로 드러난다. 논문은 자기기법을 **virtual context management** 라고 부르고 프롬프트를 main memory, 저장소를 disk 에 대응시킨다. 이 은유가 새는 자리가 셋이다. 첫째, MemGPT 의 page fault 는 비자발적 트랩이 아니라 프롬프트 안에 텍스트로 배달되는 **경고**다. 둘째, 주소 변환도 보호도 상주 보장도 없다 — 이동이 전부 자발적이다. 셋째, 논문이 인용하는 유일한 OS·아키텍처 문헌은 1988 년 Patterson·Gibson·Katz 이고, 그것은 가상 메모리 논문이 아니라 **RAID 논문**이다.

그리고 더 무거운 사실이 하나 있다. **MemGPT**에서는 **아무것도 학습되지 않는다**. gradient 가 한 번도 계산되지 않고, loss 가 없으며, 이 방법에 속하는 파라미터가 없다. 논문에 training 절 자체가 없다. 대상 모델은 동결된 상용 API 셋 (gpt-4-1106-preview 128,000 / gpt-4-0613 8,192 / gpt-3.5-turbo-1106 16,385) 뿐이다. 기억관리 정책은 학습되는 것이 아니라 시스템 프롬프트에 영어로 지시된다.

기호 풀이 “학습된다” 는 이 권에서 항상 gradient 갱신을 뜻한다. loss $\mathcal{L}$ 을 정의하고, 그 값을 줄이는 방향을 좌표마다 계산해서 ($\nabla_{\theta} \mathcal{L}$), 그 반대로 가중치를 한 걸음 옮기는 절차다. 이 절차가 한 번도 등장하지 않으면 그 논문에는 학습이 없다.

W 층도 여기서는 빈칸이다. W 가 있는 아키텍처라면 문맥이 흘러갈 때마다 고정 크기 상태를 갱신하고 원본을 버리지만, MemGPT 는 그런 층을 갖고 있지 않다. 이 계보 전체에서 W 는 끝까지 등장하지 않는다.

시스템 모델링 관점 커널 안에서는 아무것도 바뀌지 않는다. 새 GEMM 도, attention 변경도, 새 상태 텐서도 없다. 바뀌는 것은 모델 위다. 첫째, **논리적턴 하나가 forward pass 하나가 아니게 된다**. 모델이 검색을 부르면 결과가 프롬프트에 붙고 다시 추론이 돌며, 이 사슬이 모델이 멈출 때까지 이어진다. 논문의 trace 그림에서 질문 하나가 검색 네 번 + 답변 한 번, 즉 최소 다섯 사이클이고 각 사이클의 프롬프트가 직전보다 길다. 그런데 논문은 이 사이클 수를 한 번도 세지 않는다. 둘째, **재귀 요약을 큐의 0 번 자리에 두는 것은 prefix KV cache 재사용에 최악의 배치다**. flush 마다 앞자리가 다시 쓰이므로 뒤따르는 KV 전부가 무효화되고, working context 에 대한 수정도 메시지 꼬리 전체에 같은 일을 한다. 논문은 KV cache 를 한 번도 언급하지 않는다. 셋째, 그럼에도 **shared-weight batching 은 온전하다**. 어떤 파라미터도 사용자마다 움직이지 않으므로 서빙 함대는 가중치 사본 하나로 에이전트들을 평범한 채팅 세션처럼 묶는다. 사용자별 상태는 Postgres 행과 프롬프트 토큰에 살고, 둘 다 기존 스택의 일급객체다. E-경로는 긴 문맥을 가속기 상태 바이트로 사지 않고 **decode 사이클과 prefix 지역성으로 산다**.

비용 네 칸은 전부 비어 있다. B_s 는 sleep 단계 자체가 없으므로 없고, L_w 는 TTFT 도 ms / token 도 wall-clock 도 토큰 수도 달려도 논문 전체에 없으며, C_{cap} 은 working context 를

“fixed-size” 라고만 하고 그 크기를 끝내 밝히지 않는다. ρ 는 반복 통합 실험 자체가 없다.

한계 그럼에도 이 논문에 대해 단정할 수 있는 비용 명제가 하나 있다. 턴 하나가 여러 사이클이 되고 각 사이클의 프롬프트가 직전보다 기니까, **답하나당 비용은 반드시 baseline 보다 크다**. 단정할 수 없는 것은 그 배수다. 그리고 그 배수를 묻는 질문 자체가 논문에 등장하지 않는다.

실험 쪽도 짚어 둔다. ablation 이 0 건이다. 주력 벤치마크인 DMR 에서 GPT-4 는 32.1% 에서 MemGPT 를 붙여 92.5% 가 되는데, 논문 자신이 baseline 은 과거 대화의 손실 요약만 보고 MemGPT 는 전체 이력을 본다고 적는다 — 즉 방법의 효과와 **입력 접근권의 효과**를 합쳐서 잔값이다. 게다가 baseline 이 뒤집혀 있다. GPT-4 baseline 32.1% 가 GPT-3.5 Turbo baseline 38.7% 보다 낮고, 논문은 이 역전을 언급하지 않는다. 판정기는 같은 모델 계열이며 “관대하게 채점하라” 고 지시받았다. 두 번째 벤치마크인 Conversation Opener 표에는 **MemGPT 를 쓰지 않은 LLM 행이 아예 없다**.

자세한 것은 ch13 § 13.2 – § 13.7.

4. 그런데 MemGPT 는 sleep-time compute 이 아니다

이 책은 어떤 절차가 이 범주에 속하는지를 네 조건으로 판정한다.

핵심 판별식. (1) **시계** — 상태를 바꾸는 연산이 질의 도착전에 실행되는가. (2) **예산** — 그 연산이 $B_s > 0$ 을 소비하는가. (3) **대상** — $\Theta \cdot W \cdot E$ 중 어느 한 층의 상태를 실제로 바꾸는가. (4) **소비자** — 바뀐 상태가 그것을 유발한 질의가 아니라 **이후 질의**에 쓰이는가. 넷을 모두 만족해야 한다.

MemGPT 는 조건 (1) 에서 떨어진다. 이 논문의 쓰기는 전부 대화중에, 사용자 턴의 임계 경로 위에서, 답을 만드는 것과 같은 디코딩 예산으로 일어난다. 사용자는 기억 정리가 끝날 때까지 기다린다.

그럼에도 이 장을 E-경로의 출발점에 두는 이유는 둘이다. 그 쓰기의 산출물이 **이후 질의**에 쓰이고 (조건 (4) 는 만족), 다음 장의 논문이 이 계보를 자기 조상으로 지목하기 때문이다. MemGPT 는 sleep-time compute 의 구조적 조상이 되 sleep-time compute 이 아니다.

그리고 유휴 예산이 들어갈 구멍은 논문 안에 이미 뚫려 있다. 이벤트 종류를 열거하는 대목에 “정기 스케줄로 도는, 사용자 개입 없는 실행” 이 적혀 있다. 어떤 실험도 이 혹을 쓰지 않고 어떤 계산도 여기에 귀속되지 않는다. **다음 장이 한 일을 한 문장으로 줄이면, 이 혹을 실제로 실행하고 그 계산에 B_s 라는 이름을 붙인 것이다.**

자세한 것은 ch11 § 11.2, ch13 § 13.4.

5. Letta 의 Sleep-time Compute — 이 라인에 이름을 준 논문

같은 저자들이 다음에 낸 논문 (arXiv:2504.13171) 이 이 라인의 이름을 만들었다. 문제의식은 비용의 크기가 아니라 비용이 청구되는 시점이다.

논문의 관측은 이렇다. 지금의 test-time scaling 은 문제가 stateless 라고 가정한다 — 질의와 그 답에 필요한 배경 문맥이 test time 에 **함께** 도착한다는 것이다. 그런데 실제 응용은 대개 그렇지 않다. document QA, 공통 저장소 위의 coding agent, 대화 이력을 들고 있는 assistant — 셋 다 다음 사용자 입력이 오기 **전에** 이미 문맥이 확보되어 있다. 그러면 조작적 질문 하나가 남는다. **문맥에만 미리 쓴 계산은 test time 에 무엇을 사주는가.**

핵심 sleep/wake 분리. 서빙 경로를 두 국면으로 쪼갬다. **sleep** 국면은 질의 q 가 아직 도착하지 않은 구간이고 문맥만 가지고 예산 B_s 를 쓴다. **wake** 국면은 q 가 도착한 뒤이고 예산 B_t 를 쓴다. 둘이 다른 점은 셋뿐이다 — (i) 입력 가용성 (q 의 유무), (ii) 지연 제약 (wake 만 사용자 대기 시간에 묶인다), (iii) 단가 (wake 토큰이 더 비싸다고 **가정된다**). 이 셋 중 하나라도 없으면 분리는 아무것도 사주지 않는다.

기제는 지극히 단순하다. sleep 국면에서 모델이 원 문맥을 읽고 그것을 다시 쓴다. 이 재작성을 최대 10 회까지 순차로 반복하고, 모델이 “다 생각했다” 는 함수를 호출하면 멈춘다. 결과로 나온 자연어 문자열 $\hat{c}$ 를 wake 국면에서 **원 문맥 자리에 대신 끼워 넣는다**. 그것뿐이다. 논문에는 번호 붙은 수식이 하나도 없고, 기제 전체가 산문 속 화살표 세 개로 진술된다.

$\hat{c}$ 의 이름인 **learned context** 에서 “learned” 는 gradient 를 뜻하지 않는다. 이 논문에도 backward pass 가 없고, 모든 대상 모델이 폐쇄 API 이며, training·fine-tuning·adapter·optimizer 가 논문 어디에도 없다. 논문 본문이 이 점을 스스로 정확하게 쓴다 — 보통의 표현 학습은 파라미터 공간이나 activation 공간에서 일어나지만 자기들은 **자연어 공간**에서 표현을 만든다는 것이다. 이 한 문장이 이 책의 3 경로 분할에 대한 가장 좋은 근거다. parameter space 가 Θ -경로, activation space 가 W -경로, natural-language space 가 E -경로다. 그리고 그 삼분을 E -경로 저자가 직접 썼다.

비유 prefix cache 와 나란히 놓으면 차이가 즉시 보인다. prefix cache 는 “이 토큰들의 KV 를 다시 만들지 않겠다” 는 장치다. **같은 계산의 재사용**이고, 무손실이며, 방향이 없다 — 어떤 질의가 오든 같은 KV 가 유효하다. $\hat{c}$ 는 “이 문맥에서 나올 결론을 미리 내두겠다” 는 장치다. **새 계산의 선행 수행**이고, 손실적이며, 방향이 있다 — 어떤 질의를 예상했느냐에 따라 유용성이 달라진다. — **비유가 깨지는 곳:** prefix cache 가 빗나가면 손해가 0 이지만 (그냥 다시 계산한다), $\hat{c}$ 가 빗나가면 손해가 음수가 아니라 **양수**다. 빗나간 추론이 프롬프트를 길게 만든 채 매 질의마다 다시 prefill 되기 때문이다.

정규형 (U-E) 와 어긋나는 자리도 넷 있다. w_r 이 **파괴적 덮어쓰기**여서 이력도 rollback 도 없다. $S(\cdot; B_s)$ 가 단발이 아니라 **순차 사슬**이다. ret 이 **항등**이다 — index 도입 베딩도 유사도 점수도 없고 저장소 전체가 통째로 끼워진다. 그리고 E 가 **문맥 범위**이고 **휘발성**이다. 문맥 하나당 저장소 하나이고, 세션을 넘지 않으며, sleep 라운드 첨자 k 가 모든 실험에서 1 을 넘지 않는다.

자세한 것은 ch14 § 14.2 – § 14.3.

6. 상각 — 이 경로의 유일한 경제논증

E -경로가 이득이라는 주장은 전부 하나의 형태를 갖는다. sleep 비용은 **문맥당 한 번** 발생하고 wake 비용은 **질의당** 발생하니까, 한 문맥을 여러 질의가 공유하면 질의당 평균 비용이 떨어진다는 것이다.

$$C_{\text{avg}} = C_{\text{wake}} + \frac{C_{\text{sleep}}}{N_q} \quad (\text{A})$$

기호 풀이 N_q = 한 문맥을 공유하는 질의 수. C_{sleep} = 그 문맥에한 번 지불한 sleep 비용, 즉 B_s . C_{wake} = 질의 하나를처리하는 비용.

이 식이 말하는 것은 미리 태운 계산이 질의 수로 나뉘는것뿐이다. N_q 가 크면 분수 항이 사라지고, $N_q = 1$ 이면 아무것도상각되지 않아 sleep 비용이 그대로 청구된다.

여기서 손익분기가 나온다. sleep 을 쓰는 팔이 더 작은 wake 예산으로 baseline 과 같은 정확도에 도달한다고 두면, 총비용에서 이기는조건은 질의 수가 어떤 문턱 N_q^* 를 넘는 것이다.

$$N_q^* = \frac{B_s}{\kappa_{\text{cost}} (B_t - B_t^{\text{lo}})}$$

기호 풀이 B_t = baseline 이 쓰는 wake 예산. B_t^{lo} = sleep 을 쓴쪽의 (더 작은) wake 예산. κ_{cost} = wake 토큰이 sleep 토큰보다 비싼 배수.

이 식이 말하는 것은 문턱을 계산하려면 값이 셋 필요하다는 것이다— 미리 태운 양 (B_s), 아낀 양 ($B_t - B_t^{\text{lo}}$), 그리고단가비 (κ_{cost}). 셋 중 하나라도 없으면 문턱은 수가아니라 이 야기다.

자세한 것은 ch14 § 14.5.

7. 분자가 없다 — B_s 가 인쇄되지않았다

이 권에서 가장 무거운 사실이 여기 있다. **상각 논증의 분자가 이논문에 숫자로 존재하지 않는다.**

논문은 sleep-time 토큰 수도, FLOP 수도, 달러 액수도 보고하지 않는다. 본문에도, 어떤 표에도, 어떤 축에도 없다. 모든 그림의 가로축이 ‘Avg. Test Time Tokens / Question’ 이고, **sleep 비용은 자기 축을한 번도 갖지 못한다.** 보고된 것은 서수뿐이다 — 재작성 호출 상한 10 회, 병렬 궤적 개수 네 가지 ($\{1, 2, 5, 10\}$), 그리고 벤더가 정의를 공개하지않은 ‘reasoning effort’ 세 단계. 상한 10 은 실제로 몇 번 호출되었는지를말해 주지 않고, 실현된 평균 호출 수도 보고되지 않는다.

나머지 세 칸도 같다. L_w — wall-clock, TTFT, ms/token 수치가 논문에 **0** 개다. 초록의 문제 제기가 ‘high latency’ 인데, 지연은 한 번도 측정되지 않고 전부 생성 토큰이라는 대리값 위에 서 있다. C_{cap} — 상태 바이트가 없다. $|\hat{c}|$ 조차 보고되지않는다. ρ — sleep 라운

드가 모든 실험에서 정확히 1 회이므로 **미측정이 아니라 구성상 범위 밖**이다. 라운드가 하나면 라운드당 손실은 정의되지 않는다.

단가비 $\kappa_{\text{cost}} = 10$ 도 측정이 아니다. 논문이 다는 근거는 Databricks 문서 페이지를 가리키는 각주 하나이고, 이 비율을 재는 실험도 이 값을 흔들었을 때 결론이 어떻게 변하는지 보는 민감도 분석도 없다. 그런데 이 상수는 곱셈으로 결론에 들어간다 — κ_{cost} 를 1로 두면 손익분기 질의 수가 그대로 10 배로 된다.

마지막으로 비용모델 자체에 구멍이 있다. 식이 세는 것은 **생성된 토큰뿐**이고 입력 토큰 항이 어느 쪽에도 없다. 그런데 이 방법의 기제자체가 **입력을 길게 만드는 것**이다. $\hat{c}$ 는 원문맥보다 길고, 병렬 스케일링에서는 최대 10 개의 재표현이 이어붙어 **매질의마다** 다시 처리된다.

시스템 모델링 관점 이 논문의 systems 공백을 한 줄로 줄이면 이렇다 — **decode 토큰 절약을 주장하면서 경계 없는 prefill 토큰 비용을 측정하지 않는다**. 세가지가 따라 나온다. 첫째, 논문의 회계는 자기 방법의 비용도 과소평가하고자 자기 방법의 절약도 과소평가한다. 생성 토큰만 세므로 공유 prefix 의 KV 재사용을 절약으로 잡지 못하고, 동시에 길어진 prefix 의 prefill 을 비용으로도 잡지 못한다. 두 오차의 부호는 반대인데 크기를 비교할 재료가 논문에 없으므로 상쇄 여부는 판정되지 않는다. 둘째, **sleep 작업은 지연에 둔감한 배치 가능 decode 잡이다**. 서버 함대의 언어로 옮기면 이 논문의 제품은 latency-critical decode 를 background decode 로 바꾸는 것이고, 논문은 그 전환에 10:1 의 가격표를 붙였다. interactive tier 의 일을 batch tier 로 옮기는 낮익은 모양이다 — 다만 그 10:1 이 이 함대에서 측정된 값이 아니다. 셋째, **sleep 의 wall-clock 은 임계 경로밖이어도 공짜가 아니다**. sleep 쓰기는 문맥당 최대 10 회의 순차모델 턴이고, 여기에 병렬 궤적 최대 10 을 곱하면 최악의 구성은 첫 질의가 오기 전에 문맥 하나당 100 회의 모델 턴이다. 함대가 단위 시간에 문맥 몇개를 예열할 수 있는지가 이 값으로 정해진다. 논문은 실제 턴 수도 처리율도 보고하지 않는다.

한계 이 책은 비용이 침묵한 자리를 성능 주장으로 메우지 않는다. 그러므로 이 논문에 대해 “효율적” 이라고 쓰지 않는다. 쓸 수 있는 것은 “**같은 정확도에서 생성 토큰이 줄었다**” 까지다. 생성 토큰 측과 비용 측 사이에는 prefill 항, 측정되지 않은 B_s , 단언된 κ_{cost} , 그리고 미지의 N_q 분포가 놓여 있다. 넷을 통과해야 두 측이 같아진다.

자세한 것은 ch14 § 14.5, § 14.7.

8. 저자들이 스스로 적어 둔 불리한결과

헤드라인은 넷이다 — test-time compute 을 약 $5\times$ 줄이고, Stateful GSM-Symbolic 에서 최대 **13%**, Stateful AIME 에서 최대 **18%** 개선하며, 문맥당 질문이 10 개일 때 질의당 평균 비용을 최대 $2.5\times$ 줄인다.

먼저 이 숫자들이 어디서 나왔는지 고정한다. **넷 다 저자가 만든 벤치마크에서 나온 값이다**. 기존 과제를 문장 경계에서 문맥 절반과 질의절반으로 잘라 만든 Stateful GSM-Symbolic(P1 5000 예제 / P2 2500 예제), 같은 방식의 Stateful AIME(60 문항), 그리고 o3-mini 가 문항을 추가 생성한 Multi-Query GSM-Symbolic 이다. 논문 안에 기존의

stateful reasoning 벤치마크는 하나도 인용되지 않는다. 그 숫자들 뒤에 독립적인기준이 없다는 뜻이다.

Stateful AIME 60 문항에는 **문맥이 비어 있는 인스턴스가 개수미상으로 포함되어** 있고 (문제에 statement 가 하나뿐인 경우), 질의 자리를 맞추기 위한 수동 재배열이 있었다. 문맥이 비면 sleep 국면이 읽을 것이없으므로 이 방법은 구성상 아무것도 할 수 없다. 분모가 60 이므로 한 문항이 1.67 점이다. Multi-Query 쪽은 **90.9%가 모델 생성 문항**이고 (P1 은 12043 개 중 10948 개, P2 는 5497 개 중 4997 개), 생성된 답의 정확성을 검증했다는 보고는 어디에도 없다. 상각 논증의 분모 N_q 가 바로 그 문항들로 채워져 있다.

그리고 논문 자신이 적어 둔 불리한 결과가 넷이다.

표 E07-1. 논문이 본문에 직접 적은 역방향 결과.

관측	어디서	논문의 처리
높은 test-time 예산에서는 baseline 이 이긴다	Stateful GSM-Symbolic 과 SWE-Features 두 과제 모두	가설만 제시하고 시험하지 않는다. 초록은 이 레짐 제한을 달지 않는다
sleep 스케일링이 비단조다 — 병렬 쿼리 5 개가 10 개보다 대체로 낮다	13% 헤드라인이 나온 바로 그 스윙	최적점을 찾는 규칙도 임계도 제시되지 않는다
가장 강한 추론 모델 (o1) 이 거의 이득을 보지 못한다	전 모델 비교	설명이 제시되지 않는다
질의 수가 적으면 총비용에서 순손실이다	비용 그림 캡션	손익분기의존재를 인정하되 위치를 숫자로 주지 않는다

여기에 평가 설계 쪽 사실 셋을 더 붙인다. **모델 선택 기준이 측정대상과 상관된다** — 논문은 “test-time compute 을 적게 썼을 때 나쁘고키우면 개선되는 모델” 을 골랐다고 적는데, 그 기울기가 가파른 모델이 정확히 $5\times$ 수평 이동을 크게 보이게 만든다. **결과 표가 없다** — 표는 하나뿐이고 그것은 데이터셋 통계다. 모든 정확도 수치가 주석 없는 선그래프이고 오차 막대도 신뢰구간도 없다. 실행 횟수는 arm 마다 다르며 (3 회또는 10 회), 그 차이는 결과를 본 뒤 “잡음이 더 관측되어서” 조정된 것이다.

한계 이 목록이 논문의 기여를 지우지 않는다. **시간 비대칭을 이용한다는 발상 자체는 옳다.** 문맥이 질의보다 먼저 도착하는 워크로드가 실재하고, 그 시간 동안 노는 계산 자원이 실재하며, 그것을 쓰면 wake 국면의 생성 토큰이준다는 것은 관측된 사실이다. 반박되는 것은 그 관측이 아니라 **그축이 곧 비용 축**이라는 암묵 등식이다.

자세한 것은 ch14 § 14.6.

9. 프로덕션 — Mem0 · Zep · ReasoningBank

다음 세대는 실제로 배포되는 시스템들이다. Mem0(arXiv:2504.19413), Zep(arXiv:2501.13956), ReasoningBank(arXiv:2509.25140) 이 대표이고, 여기에 SCM 과 Memini 가 따라붙는다. 이들은 서로 다른 것을 고치겠다고 선언한다 — Mem0 은 **무엇을 얼마나 적게 다시 읽을 것인가**, Zep 은 **어떤 사실이 언제 참이었는지를 어떻게 기록할 것인가**, ReasoningBank 은 **경험을 어떤 형태로 저장해야 재사용되는가**를 묻는다.

방향은 셋인데 빼대는 하나다.

핵심 E-경로 프로덕션 삼단 구조. (i) 동결 LLM 이 도착한 문맥에서 후보레코드를 뽑는 **추출**, (ii) 같은 동결 LLM 이 기존 저장소와 대조해쓰기 연산자를 고르는 **통합**, (iii) 임베딩·역색인·그래프 탐색으로 후보를 뽑아프롬프트 문자열로 직렬화하는 **검색**. 학습되는 파라미터는 없고, 시스템의 실질적 '가중치'는 프롬프트와 사람이 손으로 고른 상수다.

차이는 전부 통합 단계, 즉 wr 의 모양에 있다. 이것을 가르느축은 하나면 된다 — **저장소가 줄어들 수 있는가**다. Mem0 은 후보사실마다 동결 LLM 이 ADD/UPDATE/DELETE/NOOP 중 하나를고르게 하므로 **줄어들 수 있다**. Zep 은 삭제 대신 유효 구간을 닫는방식이라 줄어들지 않는다. ReasoningBank 은 순수 합집합이라 줄어들지않는다. SCM 은 임계미달 노드를 지우므로 줄어들 수 있다.

시간을 표현하는 방식도 셋뿐이다. **명시적 타임스탬프** (Zep 은 사실이세계에서 참이었던 구간과 시스템이 그것을 알게 된 순서를 **따로**기록한다), **감쇠 상수** (SCM·Memini 는 시각을 저장하지않고 상태가 줄어드는 속도로 시간을 표현한다), **순서첨자** (ReasoningBank 은 과제 번호만 갖고 지속 시간을 아예 모형화하지않는다).

Zep 의 방식이 가장 정교한데, 여기서 이 경로의 구조적 약점이 드러난다. 이중 시간으로 기록한 유효 구간이 결국 **프롬프트 안에 문자열로들어간다** (“FACT (Date range: from - to)”). 시간 임베딩도, 검색 단계의유효성 필터링도, 시간 전용 attention 도 없다. 즉 시간을 쓰는 일이 검색의보장이 아니라 **추론자의 과제**로 전가된다. 결과가 그대로 나온다 —사실이 바뀔 때 저장소가 버티는지를 재는 knowledge-update 범주에서, 약한모델 (gpt-4o-mini) 로는 76.9%에서 74.4%로 **떨어지고** 강한모델 (gpt-4o) 로는 78.2%에서 83.3%로 오른다. **이득의 부호가 생성모델의 성능에 따라 바뀐다**.

비유 삭제 가능한 쓰기 연산자는 데이터베이스의 트랜잭션 로그 같은 것이다. 무엇을 넣고 무엇을 지웠는지의 기록 위에서 시스템이 돈다. — **비유가 깨지는 곳**: 진짜 DB 에는 되돌리기, tombstone, 감사 로그가있고 삭제 판단은 결정론적 술어가 내린다. 여기서는 셋 다 없고, 삭제판단을 동결 LLM 이 확률적으로 내리며, **두 시스템 모두 그 판단의정확도를 한 번도 측정하지 않는다**.

자세한 것은 ch15 § 15.3 – § 15.4.

10. 품질 1 위가 반복해서 “기억 시스템없음” 이다

여기가 이 권의 두 번째 결론이다. 다섯 논문의 **자기 표**를 그대로읽으면 같은 일이 다섯 번 일어난다.

표 E07-2. 각 논문의 자기 표에서 품질 1 위가 무엇인가.

논문	품질 1 위	제안 시스템	격차
Mem0	full-context 72.90 ± 0.19%	Mem0 66.88 ± 0.15%	6.02 점. 그래프 변형 (68.44) 과도 4.46 점

논문	품질 1 위	제안 시스템	격차
Zep (DMR, gpt-4-turbo)	Zep 94.8%	—	그러나 full-conversation baseline 94.4%가 이미 비교 대상 MemGPT 93.4%를 넘는다
SCM	No-Forget Graph 22/22	SCM 22/22	동물 . 삭제 기제의 품질 기여가 0 이고 기여는 전부 저장 크기 (24 대 72)
ReasoningBank	검색 경험 1 개일 때 49.7	기본값도 1 개	0 → 4 로 늘리면 39.0, 49.7, 46.0, 45.5, 44.4 — 1 을 넘으면 단조 악화
Memini	—	—	읽기 경로를 한 번도 실행하지 않았다 . 저자들이 검색 지표를 보고하지 않는다고 명시한다

격차의 크기도 재 둔다. Mem0 의 6.02 점은 보고된 ± 1 표준편차바깥에 한참 있다. 반대로 Zep 이 얻는 마진은 gpt-4o-mini 에서 **0.2%p** 인데, 대화 500 건에서 0.2%p 는 **대화 한 건**이다. 신뢰구간도, 시드도, 반복 실행도, 유의검정도 논문 어디에도 없다. 게다가 비교 상대인 MemGPT 93.4%는 직접 돌린 값이 아니라 인용값이다 — 논문이표에 출처 표시를 달고, 본문에는 gpt-4o-mini 로 MemGPT 의 결과를 재현하지 못했다고 적는다. 직접 비교는 존재하지 않는다.

논문들이 이 사실을 숨기지는 않는다. Mem0 은 본문에서 full-context 가 최고점수라고 인정하고, Zep 은 **자기 벤치마크가 부적절하다고 스스로판정한다** — 현대 LLM 의 단순 full-context 접근이 높은 성능을 내는 것 자체가 이 벤치마크가 기억 시스템 평가에 부적합함을 보여 준다는 것이다. 문제는 그 인정이 본문 안쪽에만 있다는 데 있다. Mem0 의 초록은 순서를 뒤집어 정확도 이득을 먼저 말하고, 결론은 네 범주 중 자기가 지는 open-domain 만 조용히 빼고 세 범주만 나열한다.

같은 결함이 다섯 논문에 반복된다. 쓰기 예산 B_s 미보고, 상태 용량을 바이트로 보고하지 않음, ablation 부재 또는 자기 표가 자기 캡션을 반증, 기억 없는 baseline 이 이기는 사실을 본문에서 인정하고 초록 · 결론에서 누락, 그리고 추출자 · 판정자 · 응답자가 같은 동결 모델인 닫힌루프에 외부 검증 없음. 마지막 항목은 특히 무겁다 — ReasoningBank 의 모든 쓰기를 여닫는 자기 판정기의 실제 정확도가 **72.7%**인데, 그 견고성을 재는 실험은 실제 판정기를 쓰지 않고 정답 라벨을 확률적으로 뒤집어 만든 가상 판정기를 쓴다. 그 잡음은 독립이고 대칭이지만, 실제판정기의 오류는 그렇지 않다.

비용 네 칸도 여전히 비어 있다. ρ 는 다섯 전부에서, 바이트 기준 C_{cap} 도 다섯 전부에서 없다. B_s 를 숫자로 인쇄한 논문은 **다섯 중 하나뿐**이다 (ReasoningBank 의 과제당 3748.4 토큰). 그래서 손익분기 N_q^* 를 계산할 수 있는 논문이 다섯 중 하나도 없다.

자세한 것은 ch15 § 15.2, § 15.6, § 15.8.

11. 비용 결과가 성능 결과의 자리에놓인다

먼저 분명히 해 둔다. **비용 절감은 실재한다**. Mem0 은 질의당 문맥을 26031 토큰에서 1764 토큰으로 줄이고 (14.76 배), 총 지연 p95 를 17.117 초에서 1.440 초로줄인다 (11.89 배). Zep 은 115k 를 1.6k 로줄이고 (71.9 배), end-to-end 지연을 31.3 초에서 3.20 초로줄인다. ReasoningBank 은 과제당 3748.4 토큰의 오버헤드로 WebArena 성공률을 40.5 에서 48.8 로 올린다. 전부 측정된 사실이고, 서버에서 곧바로 돈이다.

문제는 **주장의 종류가 바뀌어 있다**는 것이다. “우리 시스템이 더정확하다” 와 “우리 시스템은 훨씬 싼 값에 거의 그만큼 정확하다” 는 다른주장이다. 다섯 논문의 초록은 전부 앞의 언어로 쓰여 있고, 다섯 논문의증거는 전부 뒤의 언어로 되어 있다. 그리고 뒤의 주장을 방어하려면 싼 값쪽 — 즉 B_s — 을 세어야 하는데, 다섯 중 넷이 그것을 세지 않았다.

시스템 모델링 관점 이 결론이 서버 관점에서 정확히 무엇을 판정 불가로 만드는지 짚는다. 첫째, **비교 대상이 캐시 없는 baseline 이다**. Mem0 의 full-context baseline 은 같은 대화에 대한 약 200 개 질의마다 동일한 26k 프리픽스를 다시 prefill 한다. prefix caching 이 걸린 스택에서는 그 prefill 이 **한번만** 청구된다. 지연 절감의 얼마가 아키텍처의 몫이고 얼마가 baseline 구성의 몫인지는 이 논문으로 판정되지 않는다. Mem0 전문에 “cache” 라는단어가 한 번도 나오지 않는다. 둘째, **제안 시스템 쪽은 캐시를오히려 않는다**. Mem0·Zep 의 검색 결과는 질의마다 달라 prefix cache 로 재사용되지 않고, ReasoningBank 은 검색 항목을 시스템 지시문**맨 앞**에 붙이므로 과제 사이의 공유 프리픽스를 위치 0 에서깨뜨린다. 셋 다 논문이 다루지 않는다. 셋째, **쓰기 쪽은 메시지 속도로 도는 두 번째 추론 워크로드다**. Zep 은 메시지 하나마다 여섯에서일곱 번의 동결 LLM 호출을 돌리고, Mem0 은 메시지쌍마다 추출 한 번과 갱신 tool call 한 번을 돌린다. 쓰기 비용은 대화 트래픽에 비례하고읽기 절감은 질의 트래픽에 비례하므로, 순이득은 전적으로 N_q 에 달려있다. 그 값을 실서빙에서 보고한 논문이 이 corpus 전체에 없다. 넷째, 아무도 말하지 않는 좋은 소식이 하나 있다 — **shared-weight batching 이 그대로 성립한다**. Θ 가 동결이므로 요청별 가중치상태가 생기지 않고, 백만 명의 사용자가 한 번의 가중치를 공유하며 차이는데이터베이스가 어떤 행을 돌려주는지에만 있다. 이것이 E-경로가 Θ -경로에 대해 갖는 구조적 서빙 이점이고, 다섯 논문 중아무도 이것을 명시하지 않는다.

그래서 이 권의 결론은 “이 시스템들이 나쁘다” 가 아니다. “**이시스템들의 주장은 현재 형태로는 검증 대상이 아니다**” 이다. 검증도 반증도되지 않는 이유는 비교할 수 있는 회계단위가 없기 때문이고, 그것을 만드는일이 이 책 Part III 의 과제다.

자세한 것은 ch15 § 15.7 – § 15.8.

12. 침묵의 지도

마지막으로 이 경로가 무엇을 인용하지 않는지를 적는다. 이름을 만든 논문의참고문헌은 22 개이고, 전수 확인 결과는 이렇다 — W-경로 인용 **0 건**, Θ -경로 직접 인용 **0 건**, 생물학·수면 신경과학 인용 **0 건**, E-경로 내부 인용**정확히 1 건** (같은 저자의 MemGPT). ‘sleep’ 이라는 중심 은유에생물학 인용이 하나도 붙어 있지 않다. 그 단어는 **휴식**를 뜻하기위해 선택된 것이지 consolidation 을 뜻하기 위해 선택된 것이 아니다.

그리고 침묵은 안쪽으로도 되돌아온다. 프로덕션 다섯 중 셋은 연대상 이름을준 논문을 인용할 수 있었는데 하지 않았다. ReasoningBank 전문에 “sleep”이라는 문자열은 한 번도 나오지 않는다.

비유 같은 부품을 세 공장이 서로 모른 채 따로 만드는 상황이다. — **비유가 깨지는 곳**: 공장은 특허와 원가로 자기가 중복했다는 사실을결국 알게 되지만, 여기서는 아무도 모른 채 끝난다. 실제로 SCM은 외부그래프의 간선 강도에 상수 배율을 곱해 오래된 것을 지우는데, 그 형식은 W 층 갱신식의 **retention gate**와 대수적으로 같다 — 새 것을 더하기 전에 옛 상태에 1보다 작은 수를 곱해 남길 비율을 정하는 장치이며, W -경로에서 이미 정리되어 있던 것이다. 침묵의 대가는 인용 예절이아니라 **중복 노동**이다.

자세한 것은 ch14 § 14.8, ch15 § 15.8.

요약

- E-경로는 세 층 중 E 만 건드린다. Θ 는 동결이고 W 는 이제보 어디에도 존재하지 않으며, 이 권의 논문 전체에 gradient가 하나도없다. 학습 가능해 보이는 모든 것은 셋 중 하나다 — 남이 사전학습해동결한 모델, 사람이 손으로 쓴 프롬프트, 그 동결 모델이 뱉은 텍스트.
- MemGPT는 저장소를 운영하는 주체를 처음으로 모델로 만들었고, 쓰기의촉발 조건을 질의에서 **용량**으로 옮겼다. 그러나 쓰기가 대화중 임계 경로 위에서 일어나므로 판별식 조건 (1)을 만족하지않는다. 구조적 조상이되 sleep-time compute은 아니다.
- MemGPT의 OS은유는 세 곳에서 샌다 — page fault가 자발적 경고이고, 주소변환·보호·상주 보장이 없으며, 인용된 유일한 OS 문헌이 RAID 논문이다.
- Letta의 Sleep-time Compute이 이 라인의 이름을 만들었다. 기제는 (U-E) 하나이고, w_r 은 파괴적 덮어쓰기, $S(\cdot; B_s)$ 는 최대 10회 순차 자기 재작성, ret 은항등, E 는 문맥과 함께 죽는다. 'learned context'의 learned는 gradient를 뜻하지 않으며 논문 본문이 그 점을 정확히 쓴다.
- **그 논문은 비용 네 값을 하나도 보고하지 않는다.** B_s 는 서수손잡이만 있고, L_w 는 wall-clock·TTFT 수치가 0개이며, C_{cap} 은 $|\mathcal{C}|$ 조차 없고, ρ 는 라운드가 1회뿐이라 구성상 범위 밖이다. 상각 논증의 분자가 없으므로 손익분기 N_q^* 가 계산되지 않는다.
- 비용모델은 생성 토큰만 세는데 이 방법의 기제 자체가 입력을 길게 만드는것이고, 단가비 10은 벤더 문서 인용이며 민감도 분석이 없다. 결과적으로 **decode 절약을 주장하면서 경계 없는 prefill 비용을 측정하지않는다.**
- $5 \times 13\% \cdot 18\% \cdot 2.5 \times$ 는 전부 저자제작 벤치마크의 값이다. Stateful AIME 60 문항에 문맥이 빈 인스턴스가개수 미상으로 섞여 있고, Multi-Query의 90.9%가 모델 생성 문항이며생성 답의 검증은 보고되지 않았다. 이득은 높은 예산에서 두 과제 모두역전하고, sleep 스케일링은 비단조다.
- 프로덕션 시스템의 뼈대는 추출→통합→검색 삼단이고, 갈리는 곳은 전부통합 단계다. 저장소가 줄어들 수 있는 시스템은 둘뿐이며 **둘 다삭제 판단의 정확도를 측정하지 않는다.**
- 자기 표에서 품질 1위가 반복해서 기억 시스템 없는 구성이다 —full-context 72.90% 대 Mem0 66.88%, full-conversation 94.4%가 이미 MemGPT 93.4%를 넘

음, No-Forget Graph 22/22 대 SCM 22/22, ReasoningBank 은 검색경험이 1 을 넘으면 단조 악화, Memini 는 읽기를 한 번도 돌리지 않았다.

- **비용 절감은 실재한다** — 26031 $\rightarrow$ 1764 토큰, 115k $\rightarrow$ 1.6k, p95 17.117 초 $\rightarrow$ 1.440 초. 실재하지 않는것은 그 절감이 성능 우위라는 주장이다. 주장의 종류가 바뀌었고, 바뀐주장을 방어하려면 세지 않은 쪽인 B_s 를 세야 한다.
- E-경로는 shared-weight batching 을 그대로 보존한다. 이것이 Θ -경로 대비 결정적 서빙 이점이며, 이 권의 논문 중 아무도 명시하지 않는다.

자가 점검

- 세 층 $\Theta \cdot W \cdot E$ 를 크기가 아니라소유자와 수명으로 가를 수 있고, 왜 E-경로가 배포 아 티팩트를 바꾸지않는지 한 문장으로 말할 수 있다.
- 판별식 네 조건을 열거하고, MemGPT 가 어느 조건에서 떨어지는지 그리고그럼에도 계보의 출발점인 이유 둘을 델 수 있다.
- “생성 토큰이 5 $\times$ 줄었다” 와 “비용이 5 $\times$ 줄었다” 사이에 놓인 네 가지 —prefill 항, 미보고 B_s , 단언된 단가비, 미지의 N_q 분포 — 를열거할 수 있다.
- 어떤 논문의 비용 칸이 비어 있을 때 쓸 수 있는 문장 (“생성 토큰이줄었다”) 과 쓸 수 없는 문장 (“효율적이다”) 을 가를 수 있다.
- 표 E07-2 의 다섯 행 중 셋을 수치와 함께 인용하고, 그 사실이 왜 각논문의 본문에는 있고 초록에는 없는지 설명할 수 있다.
- Zep 의 knowledge-update 가 약한 모델에서 떨어지고 강한 모델에서 오르는이유를, 유효 구간이 프롬프트 문자열로 전달된다는 사실로 설명할 수있다.
- (Rosetta) prefix cache 와 $\hat{c}$ 의 차이를 재사용의 종류로설명할 수 있다 — 같은 계산 의 재사용 대 새 추론의 선행 수행. 그리고 E-경로가 그 대가로 캐시 재사용을 어디서 잃 는지 (질의마다 달라지는 검색결과, 위치 0 에 붙는 검색 항목) 두 사례로 말할 수 있다.

다음 권으로

이 권은 하나를 열어 둔 채 끝난다. 여기서 $\text{ret}(E, q)$ 의산출물은 언제나 **문자열**이었다. Zep 의 유효 구간도, ReasoningBank 의 전략도, Mem0 의 기억도 전부 프롬프트에 실리는 토큰이된다. 그래서 E-경로의 읽기 비용은 **검색된 바이트가 곧 prefill 토큰이 되는** 형태이고, 항목 수를 늘리면 곧바로 손해가 난다 (검색 경험 1 개 49.7 $\rightarrow$ 4 개 44.4).

다음 권은 층을 옮긴다. 같은 문맥을 문자열이 아니라 **파라미터모양의 산출물**로 바꾸면 어떻게 되는가. 옮겨 가는 질문은 셋이다. 이 권의쓰기 연산자 분류 중 무엇이 그 층에서 살아남는가, 세 가지 시간 표현 중무엇이 표현 가능한가, 그리고 가장 중요하게 — **비용 결과가 성능결과의 자리에 놓이는 이 패턴이 거기서도 반복되는가.**